EJERCICIOS UNIDAD 3

Contenidos:

- Uso de bucles while, do-while y for.
- Uso básico de cadenas

Crea un nuevo proyecto independiente para cada uno de estos ejercicios. En todos los proyectos crea un comentario en el que aparezca tu nombre completo.

Realiza una captura de pantalla de tu solución y pégala en el documento.

⚠️ **Para la realización de estos ejercicios queda completamente prohibido utilizar ninguna herramienta externa, salvo la documentación oficial de Java.**

Ejercicios para resolver con bucles while:

Ejercicio 1: Vamos a crear una aplicación que pida un número del 1 al 100 al usuario. A continuación, mostrará ese número de veces el saludo: "Buenos días!".

Ejercicio 2: Crea un programa que le pida al usuario un número entero de inicio de un contador y un número de fin del mismo contador. A continuación, muestre todos los números desde el inicio al final sin incluir los dos extremos indicados. Finalmente mostrará un mensaje indicando: "se han mostrado *N* números" siendo N la cantidad de números mostrados.

Ejercicio 3: Diseña una aplicación que muestre la edad máxima y mínima de un grupo de alumnos. El usuario introducirá las edades y terminará escribiendo un -1 para indicar que no quiere añadir más edades.

Ejercicio 4: Crea un programa que muestre, para cada número introducido por teclado, si es par, si es positivo y su cuadrado. El proceso se repetirá hasta que el número introducido sea 0.

Ejercicio 5: Implementa una aplicación para calcular datos estadísticos de las edades de los alumnos de un centro educativo. Se introducirán datos hasta que uno de ellos sea negativo, y se mostrará: la suma de todas las edades introducidas, la media, el número de alumnos y cuántos son mayores de edad.

Ejercicio 6: Codificar el juego “el número secreto”, que consiste en acertar un número entre 1 y 100 (generado aleatoriamente). Para ello se introduce por teclado una serie de números, para los que se indica: «mayor o «menor, según sea mayor o menor con respecto al número secreto. El proceso termina cuando el usuario acierta o cuando se rinde (introduciendo un -1).

AYUDA: Mirar en la clase Math como se puede generar un número aleatorio.

Ejercicio 7: Un centro de investigación de la flora urbana necesita una aplicación que muestre cuál es el árbol más alto. Para ello se introducirá por teclado la altura (en centímetros) de cada árbol (terminando la introducción de datos cuando se utilice -1 como altura). Los árboles se identifican mediante etiquetas con números únicos correlativos, comenzando en O. Diseñar una aplicación que resuelva el problema planteado.

Ejercicio 8: Vamos a crear un programa que nos ayude a agilizar nuestra mente con las operaciones de sumar. Para ello, diseñaremos un programa que nos vaya mostrando números aleatorios entre el 1 y el 100 (ambos incluidos). Al finalizar nos preguntará si sabemos cuál es el resultado final de la suma de todos los números que nos ha mostrado, dejando que introduzcamos un valor. Si lo acertamos, nos dirá :”Enhorabuena!! Eres un crack matemático” y si fallamos, nos dirá “Fallaste, el resultado es N”.
El funcionamiento del programa será el siguiente:

1. El programa nos mostrará un número aleatorio
2. El programa nos preguntará si queremos otro número (pulsa cero) o si quieres finalizar (pulsa 1).
3. Si pulsamos cero volveremos al punto 1.
4. Si pulsamos uno nos mostrará un mensaje preguntándonos por el resultado.
5. Finalmente nos indicará si hemos acertado o no.

Ejercicios para resolver con bucles do-while:

Ejercicio 9: Desarrollar un juego que ayude a mejorar el cálculo mental de la suma. El jugador tendrá que introducir la solución de la suma de dos números aleatorios comprendidos entre 1 y 100. Mientras la solución introducida sea correcta, el juego continuará. En caso contrario, el programa terminará y mostrará el número de operaciones realizadas correctamente.

Ejercicio 10: Ahora que somos expertos programadores, vamos a crear un programa que nos ayude a ahorrar. Para ello, el usuario introduce su meta de ahorro (por ejemplo, 100 €).

Una vez introducida, le va pidiendo al usuario la cantidad que va ahorrando y que no puede tocar. El programa estará activo hasta que el usuario logre superar la meta establecida. Para ello puede mostrar los siguientes mensajes:

- Si ha alcanzado la meta exacta: “¡Objetivo cumplido!”
- Si la ha superado: “¡Te has pasado, ahorrador experto!”
- Si se quedó corto: “Sigue intentándolo.”

Al finalizar mostrará el número de depósitos realizados.

🔥 **Ejercicio 11:** Vas a desarrollar una aplicación en Java que permita al usuario crear su propia pizza personalizada eligiendo los ingredientes que desee.

El programa mostrará una lista con 7 ingredientes disponibles y su precio unitario.

El usuario podrá añadir ingredientes uno a uno, y después de cada elección el programa le preguntará:

¿Quieres añadir otro ingrediente? (pulsa 0 si es que si, pulsa 1 si deseas finalizar)

Al finalizar, el programa mostrará:

El número total de ingredientes seleccionados.

El precio total de la pizza, calculado sumando el coste de los ingredientes elegidos.

Un mensaje final que indique el tipo de pizza según el número de ingredientes:

🍞 Sencilla → menos de 3 ingredientes

🍕 Completa → entre 3 y 5 ingredientes

🍕🔥 Súper pizza → más de 5 ingredientes

El programa debe tener en cuenta que una pizza debe tener al menos un ingrediente.

Un posible listado de ingredientes , junto con sus precios, puede ser:

Nº	Ingrediente	Precio (€)
1	Queso extra	1.50 €
2	Jamón york	1.20 €
3	Champiñones	1.00 €
4	Bacon	1.40 €
5	Aceitunas negras	0.80 €
6	Cebolla caramelizada	1.10 €
7	Peperoni picante	1.60 €

Un posible reto adicional, para hacer el programa mejor, sería tener en cuenta que no se pudieran repetir los ingredientes.

Ejercicios para resolver con for:

Ejercicio 12: Realiza una aplicación que pida al usuario un número entero positivo y muestre en pantalla tantos "Eco" como haya introducido. Se debe realizar la comprobación de que el usuario ha introducido un número válido.

Si por ejemplo introduce el número 4 se mostrará:

Eco..
Eco..
Eco..
Eco..

Ejercicio 13: Escribir una aplicación para aprender a contar, que pedirá un número n y mostrará todos los números del 1 a n.

Opción avanzada: ¿Has tenido en cuenta que el número introducido puede ser negativo? Si por ejemplo el usuario introduce -3, si queremos mostrar de 1 a n tendríamos como resultado: 1 0 -1 -2 -3

Ejercicio 14: Escribir todos los múltiplos de 7 menores que 1000. Aprovecha el funcionamiento del for para realizar este programa de forma sencilla.

Ejercicio 15: Crea un programa que pida 8 números enteros por teclado y muestra la media.

Ejercicio 16: Realiza un programa que muestre la suma de los 30 primeros números impares.

Nota: Si se ha hecho correctamente el ejercicio el resultado debe ser 900.

¿Sería complicado modificar el ejercicio para pedirle al usuario cuantos números impares quiere?

Ejercicios para resolver con los bucles vistos hasta ahora:

🔥**Ejercicio 17:** Implementa un programa que pida al usuario un número positivo y lo muestre cifra a cifra. Por ejemplo, para el número 123, debe mostrar primero el ,3 a continuación el 2 y por último el 1.

Ejercicio 18: Crea un programa que pida un número y calcule su factorial. Por ejemplo, el factorial de 5 se denota 5! y es igual a 5 x 4 x 3 x 2 x 1 = 120.

Ejercicio 19: Implementar una aplicación que pida al usuario un número comprendido entre 1 y 10. Hay que mostrar la tabla de multiplicar de dicho número, asegurándose de que el número introducido se encuentra en el rango establecido.

Ejercicio 20: Diseña un programa que pida las notas de 7 alumnos, escribir un resumen final con la cantidad de alumnos aprobados, en duda (nota igual a cuatro) y suspensos.

Ejercicios para resolver con anidación de bucles y/o salidas anticipadas:

Ejercicio 21: Diseña una aplicación que muestre las tablas de multiplicar del 1 al 10 utilizando anidación de bucles.

Ejercicio 22: Diseña una aplicación que ahora muestre tablas de multiplicar de forma progresiva, esto es que cada tabla de multiplicar que muestre solo llega hasta su propio número:

```
1 x 1 = 1
---
2 x 1 = 2
2 x 2 = 4
---
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
...
```

🔥**Ejercicio 23:** Crea un programa que pida un número entero por consola y dibuje un triángulo de n elementos de lado utilizando asteriscos. Por ejemplo, para n=4;

```
* * * *
* * *
* *
*
```

Ejercicios para trabajar cadenas:

Ejercicio 24: Diseña un programa que permita introducir por teclado dos frases e indicar cuál de ellas es la más corta, es decir, la que contiene menos caracteres.

Ejercicio 25: Diseñar el juego «Acierta la contraseña». La mecánica del juego es la siguiente: el primer jugador introduce la contraseña; a continuación, el segundo jugador debe teclear palabras hasta que la acierte. El programa mostrará la longitud de la contraseña mediante tantos asteriscos como caracteres tenga. Según introduzca palabras el jugador, se irá comprobando si el orden de algún carácter de

la palabra introducida es correcta o no y en ese caso irá mostrando los caracteres que haya acertado.
Cuando el jugador acierte la palabra, se le dará la enhorabuena y se indicará el número de intentos que ha necesitado.
Un ejemplo de uso:

```
run:
Jugador 1, introduce la contraseña secreta: casa

La contraseña tiene 4 caracteres.
Pista: ****

Jugador 2, intenta adivinar: mosa
Pista: **sa

Jugador 2, intenta adivinar: cosa
Pista: c*sa

Jugador 2, intenta adivinar: cAsa
Pista: c*sa

Jugador 2, intenta adivinar: casa
Pista: casa

🎉 ¡Enhorabuena! Has acertado la contraseña "casa"
🔢 Intentos necesarios: 4
BUILD SUCCESSFUL (total time: 40 seconds)
```

Ejercicios varios con los contenidos del tema:

🔥 **Ejercicio 26:** Diseña un programa que pida un alto y un ancho y pinte un rectángulo cuyos bordes sean asteriscos y el interior puntitos. Por ejemplo, alto=4 y ancho=8

```
********
*......*
*......*
********
```